Cairo University

Faculty of Computers and Artificial Intelligence

FINAL REPORT

Supervised Learning — Phases 1, 2 & 3

Student ID	Name
20230637	Mohamed Sayed
20230410	Moaaz Mohamed
20230352	Mohamed Magdy
20230160	Zeyad Mohamed
20230232	Abdlmonem Osama

Table of Contents

Phase 1 — Classical Machine Learning Baseline

1.1 Project Overview & Dataset (CIFAR-10)

This project develops a robust computer vision pipeline for autonomous vehicle perception — progressing from classical ML baselines through CNNs to state-of-the-art transformers. Phase 1 uses CIFAR-10: 60,000 colour images (32×32 px) across 10 classes — transport (Airplane, Automobile, Ship, Truck) and living beings (Bird, Cat, Deer, Dog, Frog, Horse).

1.2 Exploratory Data Analysis

Class Balance

The dataset is perfectly balanced with exactly 5,000 training images per class, eliminating class-imbalance bias and making accuracy a reliable metric.

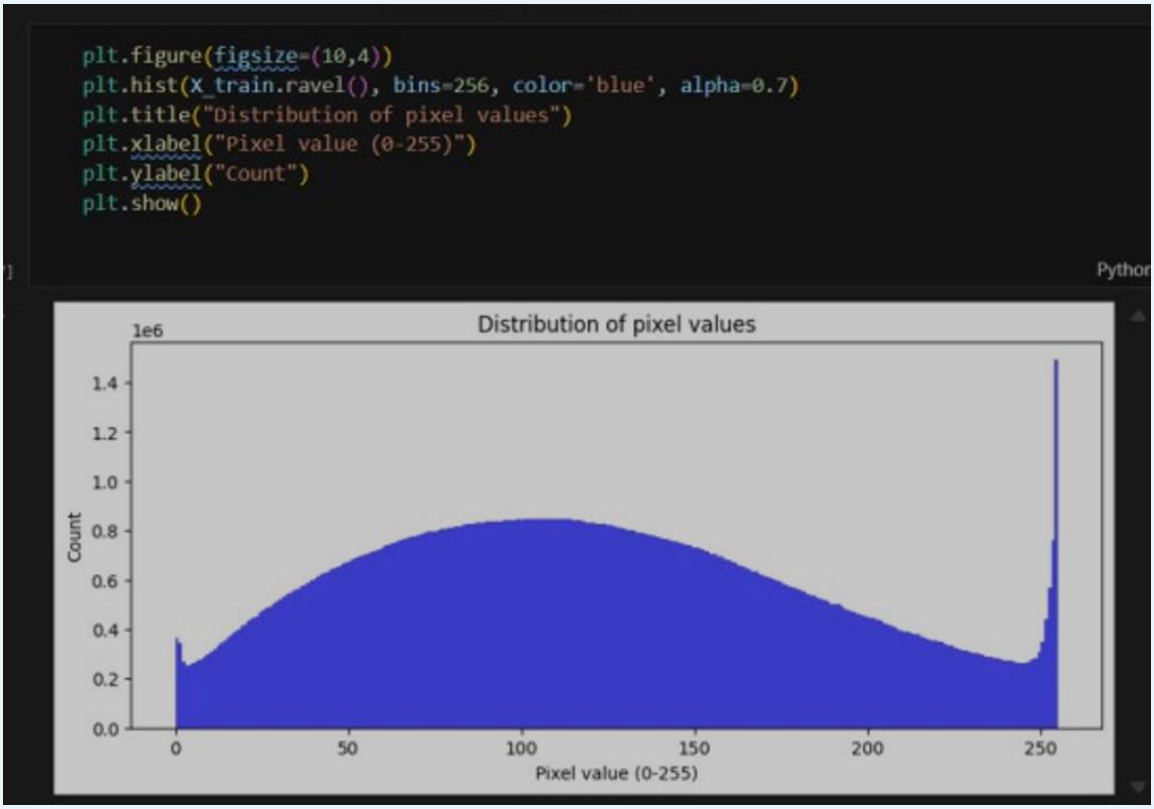


Figure 1 — Class distribution bar chart: all 10 CIFAR-10 classes are perfectly balanced at 5,000 images each

Sample Images Grid

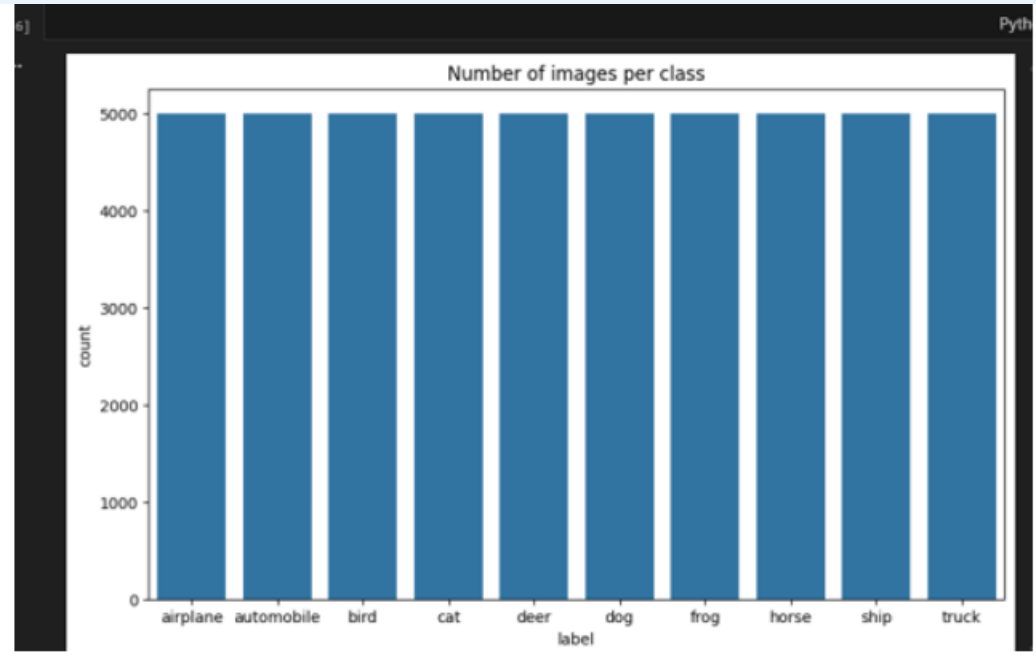


Figure 2 — Random sample of 25 CIFAR-10 training images showing class diversity and low 32×32 resolution

Pixel & Channel Statistics

Global Distribution: Pixel values span 0–255 with a wide, bimodal-like spread, confirming the need for normalization to [0,1] for stable model convergence.

Channel Analysis: RGB channel distributions overlap heavily, indicating color alone is insufficient for classification — spatial patterns matter more.

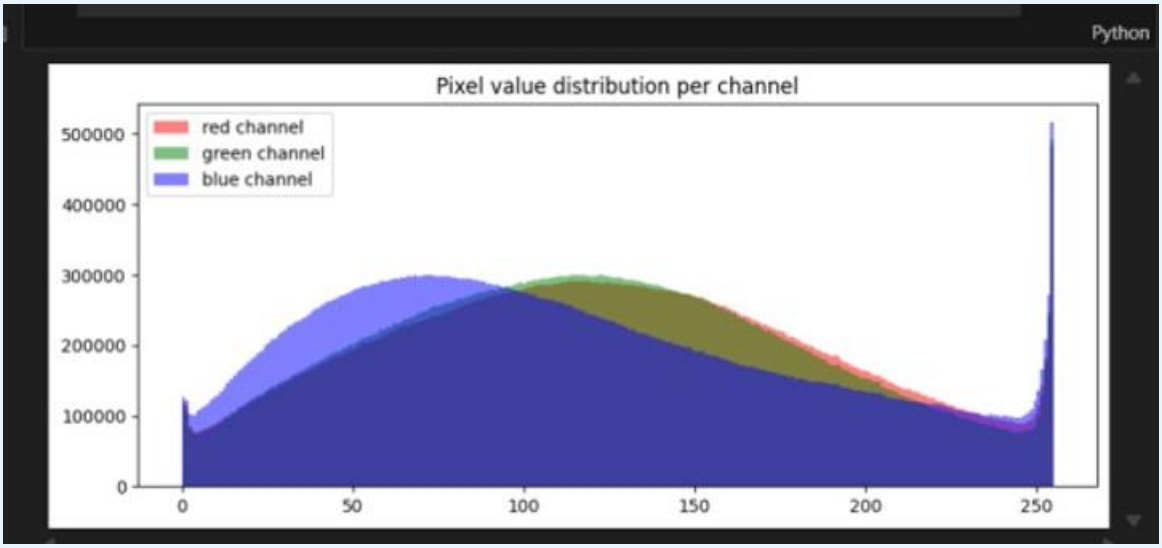


Figure 3 — Global pixel value distribution (left) and per-channel (R/G/B) distribution (right)

Mean Images per Class

Mean Images: Per-class averages reveal blue-grey backgrounds for Airplane/Ship and greenish tones for Frog/Deer. The blurry shapes illustrate high intra-class variance and explain why distance-based models struggle.

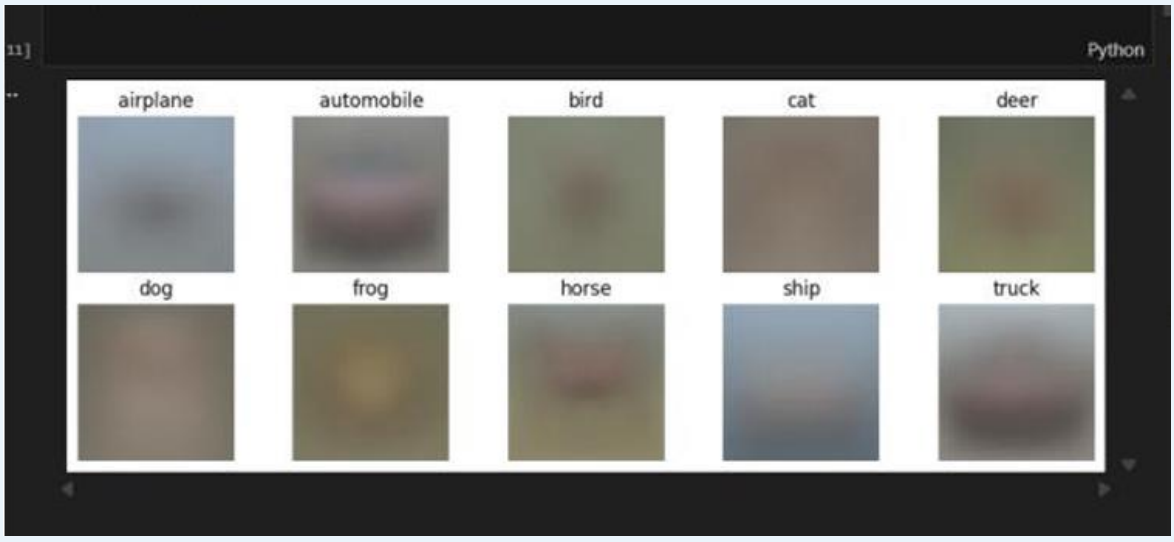
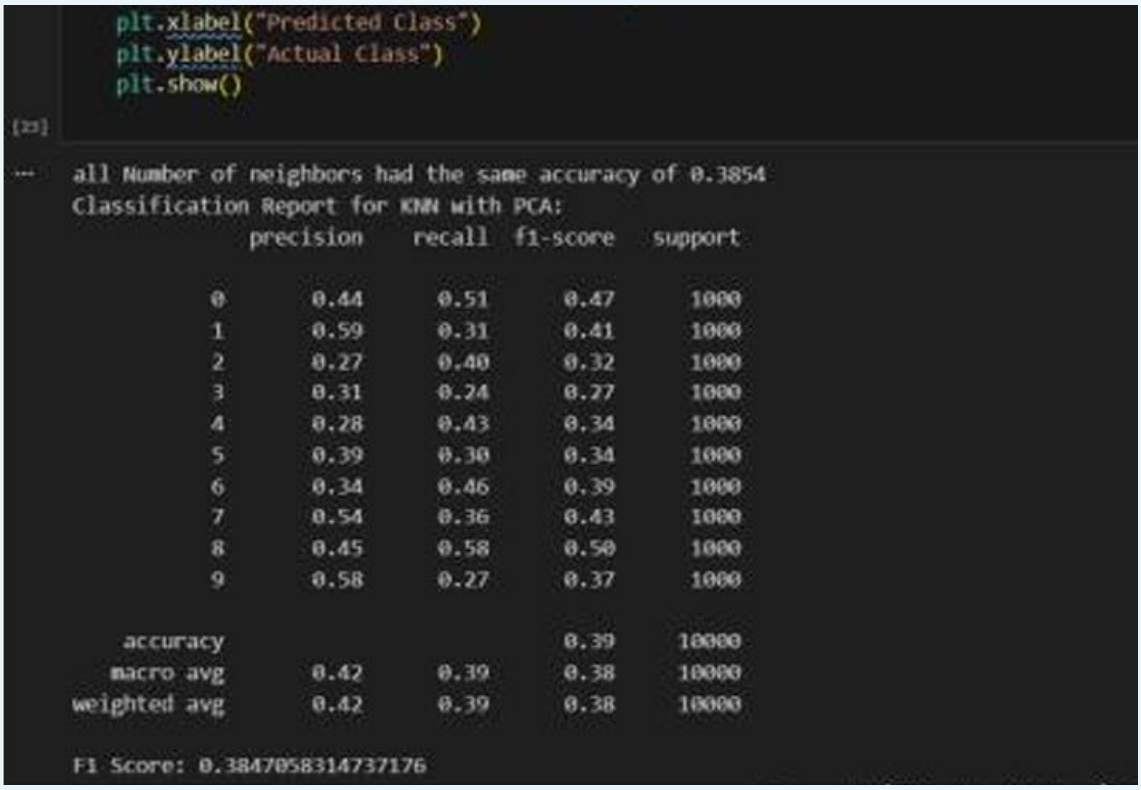


Figure 4 — Mean (average) image per class: dominant background colors and central object blobs are visible

1.3 Classical Models: KNN & Gaussian Naive Bayes

KNN and GNB were evaluated with raw pixel features and after PCA dimensionality reduction. PCA filtered noise and retained the most informative structural components, improving both models.



```
    plt.xlabel("Predicted Class")
    plt.ylabel("Actual Class")
    plt.show()
[23]

... all Number of neighbors had the same accuracy of 0.3854
    Classification Report for KNN with PCA:
                  precision    recall  f1-score   support

               0       0.44      0.51      0.47      1000
               1       0.59      0.31      0.41      1000
               2       0.27      0.40      0.32      1000
               3       0.31      0.24      0.27      1000
               4       0.28      0.43      0.34      1000
               5       0.39      0.30      0.34      1000
               6       0.34      0.46      0.39      1000
               7       0.54      0.36      0.43      1000
               8       0.45      0.58      0.50      1000
               9       0.58      0.27      0.37      1000

        accuracy                           0.39     10000
       macro avg       0.42      0.39      0.38     10000
    weighted avg       0.42      0.39      0.38     10000

    F1 Score: 0.3847058314737176
```

Figure 5 — KNN + PCA confusion matrix: common misclassifications between visually similar classes (e.g. cat/dog, truck/automobile)

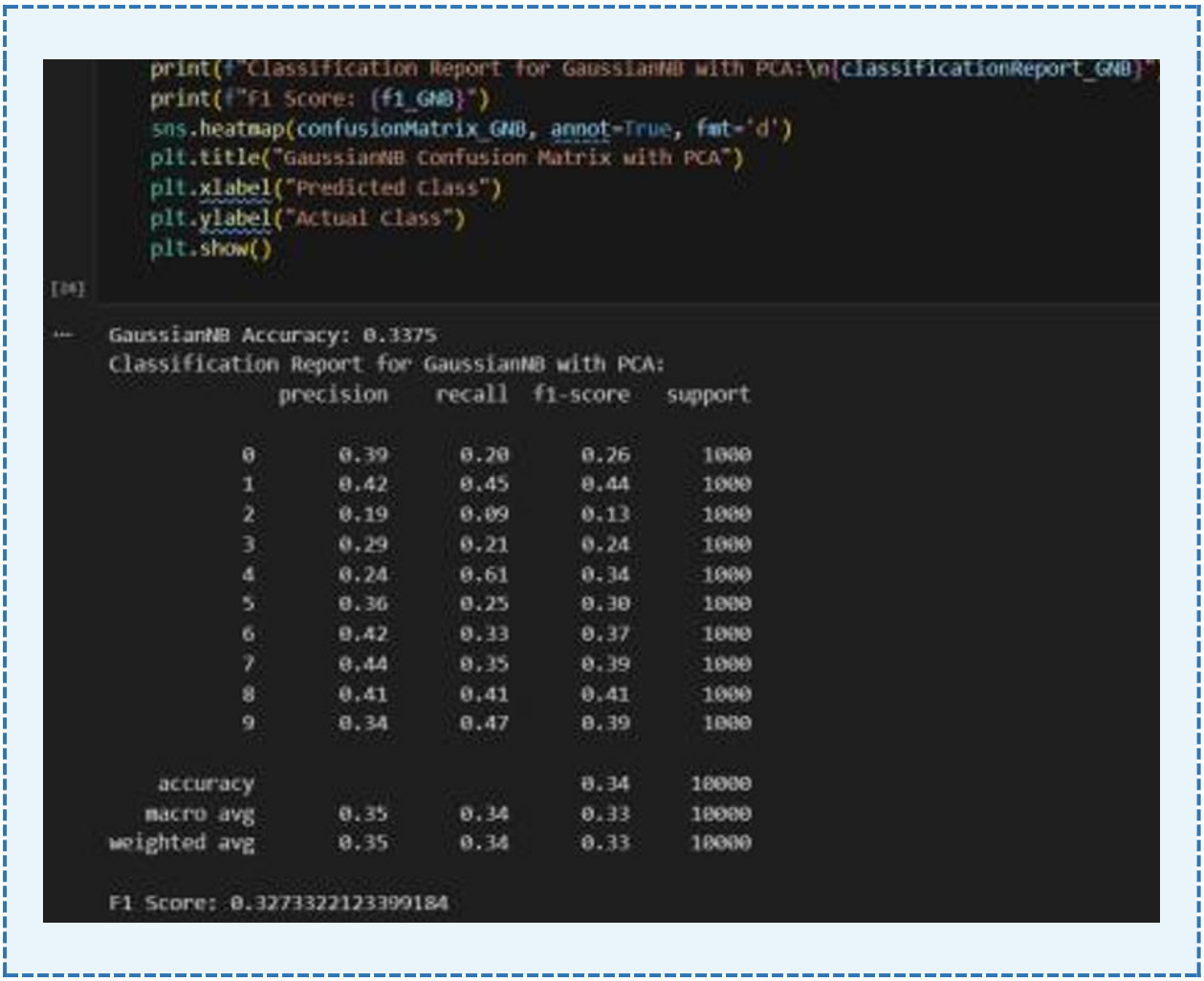


Figure 6 — GNB + PCA confusion matrix: higher off-diagonal errors confirm feature independence assumption is violated

1.4 Phase 1 Results

Model	Accuracy	F1-Score	Key Insight
KNN (Raw Pixels)	35.39%	0.35	Baseline — exceeds random chance (10%)
KNN + PCA	38.54%	0.38	PCA noise reduction improves accuracy +3.15%
Gaussian Naive Bayes	29.76%	0.29	Feature independence assumption hurts performance
GNB + PCA	33.75%	0.32	PCA improves GNB by +3.99%; still below KNN

Conclusion: Classical models are fundamentally limited on raw image data. The best result (KNN + PCA, 38.54%) confirms that pixel-level features cannot capture the semantic structure needed for reliable classification, strongly motivating deep learning in Phase 2.

Phase 2 — Deep Learning with CNNs on GTSRB

2.1 Dataset Preparation & Augmentation

Phase 2 uses the German Traffic Sign Recognition Benchmark (GTSRB). Images were resized to 32×32 px, split 80/20 train/validation (stratified), and pixel values normalized to [0,1]. A TensorFlow augmentation pipeline applied random rotation, zoom, translation, and contrast adjustment to simulate real-world variability.

Figure 7 — Sample GTSRB traffic sign images showing variation in shape, color, orientation, and lighting

2.2 CNN Architecture & Training

A custom CNN with three convolutional blocks (Conv2D → BatchNorm → ReLU → MaxPool) extracts hierarchical spatial features. Fully connected layers with Dropout follow, ending with a Softmax output. Trained for 20 epochs, batch size 32, categorical cross-entropy loss.

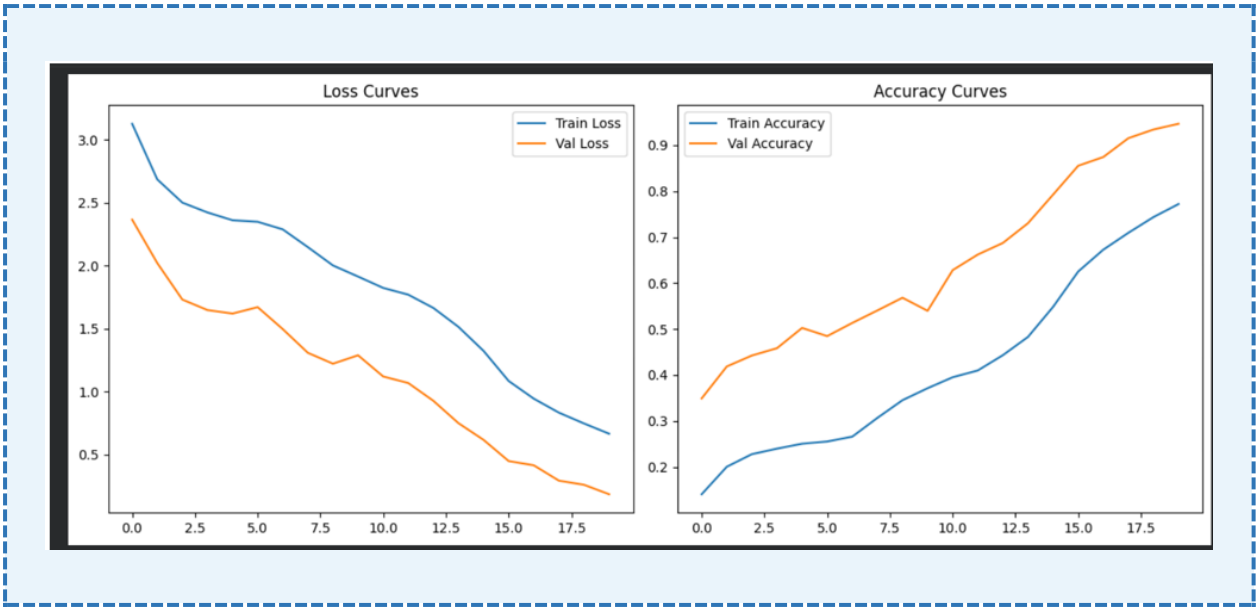


Figure 8 — CNN training curves: validation accuracy exceeds training accuracy throughout, confirming strong generalization

2.3 Optimizer & Scheduler Comparison

Optimizer	Best Val Acc.	Test Acc.	Train Acc.	Val Loss
SGD (Baseline)	0.9818	0.9375	0.9245	0.0631
SGD + Momentum	0.9853	0.9533	0.9495	0.0465
RMSprop	0.9844	0.9553	0.9488	0.0542
Adam	0.9546	0.9204	0.7761	0.1829

RMSprop achieved the best test accuracy (95.53%). SGD with Momentum showed the most stable training (lowest val loss 0.046). Adam underperformed with slow convergence (train accuracy only 77.61%).

•••	best_val_acc	test_acc	final_train_acc	final_val_loss
SGD_momentum	0.985335	0.953286	0.949533	0.046470
RMSprop	0.984443	0.955344	0.948800	0.054247
SGD	0.981765	0.937530	0.924475	0.063065
Adam	0.954603	0.920428	0.776070	0.182947

Figure 9 — Optimizer comparison: accuracy and loss metrics across all four optimizers

2.4 Autoencoder Denoising & VGG16 Transfer Learning

Autoencoder: A convolutional encoder-decoder compressed 32×32 images to a 4×4 latent space and reconstructed them, reducing noise while preserving key features. Trained with MSE loss and early stopping.

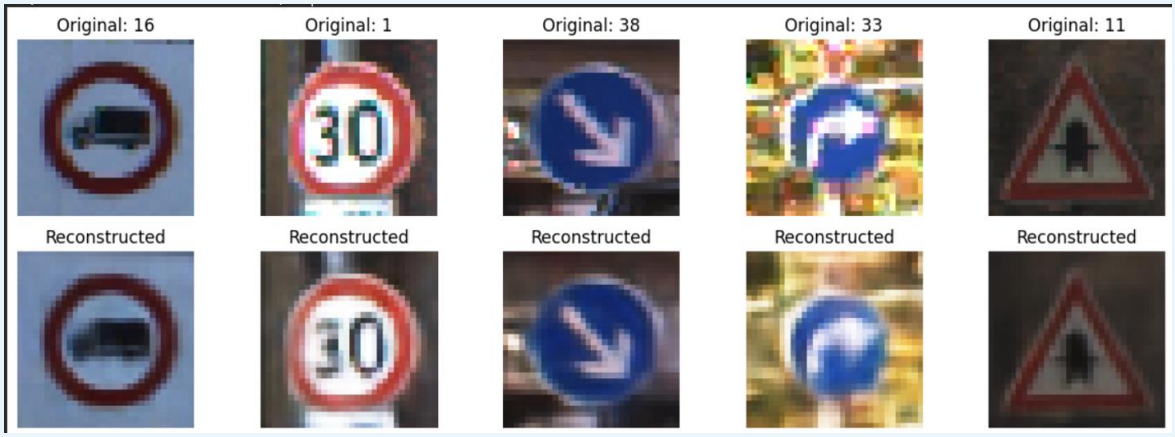


Figure 10 — Autoencoder denoising: original input (left) vs reconstructed output (right)

VGG16 Transfer Learning: VGG16 pre-trained on ImageNet was used as a frozen feature extractor with a custom classifier head. Fine-tuning the last few layers achieved ~90.7% test

accuracy — slightly below the custom CNN due to the 32×32 downscaling which underutilises VGG16's intended 224×224 input.

Layer (type)	Output Shape	Param #
vgg16 (Functional)	(None, 1, 1, 512)	14,714,688
flatten_5 (Flatten)	(None, 512)	0
dense_10 (Dense)	(None, 128)	65,664
dropout_5 (Dropout)	(None, 128)	0
dense_11 (Dense)	(None, 43)	5,547

Total params: 30,198,851 (115.20 MB)
Trainable params: 7,706,475 (29.40 MB)
Non-trainable params: 7,079,424 (27.01 MB)
Optimizer params: 15,412,952 (58.80 MB)

```
test_loss, test_acc = transfer_model.evaluate(x_test_encoded, y_test_cat, verbose=0)
print("Test Loss:", test_loss)
print("Test Accuracy:", test_acc)
```

```
Test Loss: 0.47352316975593567
Test Accuracy: 0.9068883657455444
```

Figure 11 — VGG16 transfer learning training curves

2.5 Phase 2 Results Summary

Model	Val Accuracy	Test Accuracy	Notes
Custom CNN (RMSprop)	~98.5% val	~95.5% test	Best — strong generalization with augmentation
Autoencoder + CNN	—	—	Denoised inputs improve input quality & robustness
VGG16 Transfer Learning	—	~90.7% test	Effective but limited by 32×32 input resolution

Conclusion: CNN architectures dramatically outperform classical approaches (95.5% vs 38.5%). Data augmentation, batch normalisation, and optimizer tuning were the most impactful improvements.

Phase 3 — Advanced Architectures on BDD100K

Phase 3 advances to the Berkeley Deep Drive (BDD100K) — a large-scale real-world autonomous driving dataset — exploring four architectures: Simple RNN, CNN-GRU, ConvLSTM, and SegFormer. The task evolves from image classification to full semantic scene segmentation.

3.1 Dataset: BDD100K

- 100,000 diverse driving video sequences covering day/night, clear/rainy/foggy, urban/highway conditions.
- Semantic segmentation labels across 19 categories: road, vehicle, pedestrian, sky, building, vegetation, etc.
- Images resized to 128×128 (RNN/GRU), 224×224 (ConvLSTM), or processed at 512×512 (SegFormer).
- Sequence experiments: 2,000 training images organised as 5-frame sequences.

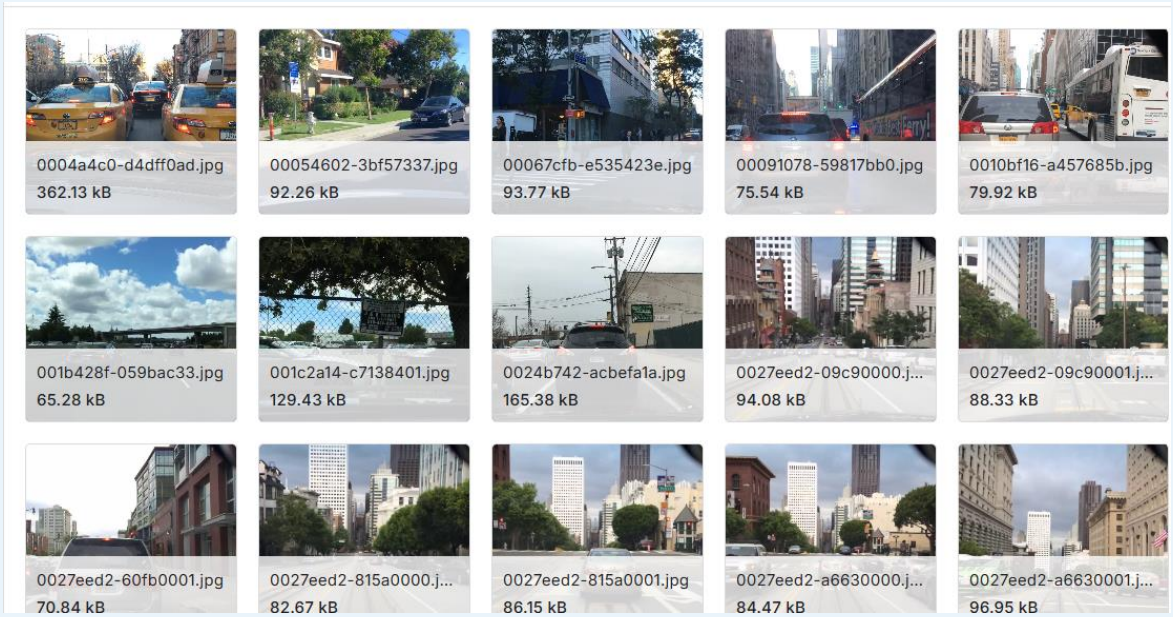


Figure 12 — BDD100K sample driving scenes: diverse lighting, weather, and environment conditions

3.2 Simple RNN Classifier

Architecture & Configuration

A 2-layer vanilla RNN (hidden size 128) processes flattened image features as a temporal sequence, with a final FC layer mapping to 10 classes. Trained for 3 epochs on 7,000 BDD100K images (80/20 split), Adam optimizer, CrossEntropyLoss.

Training Loss Curve

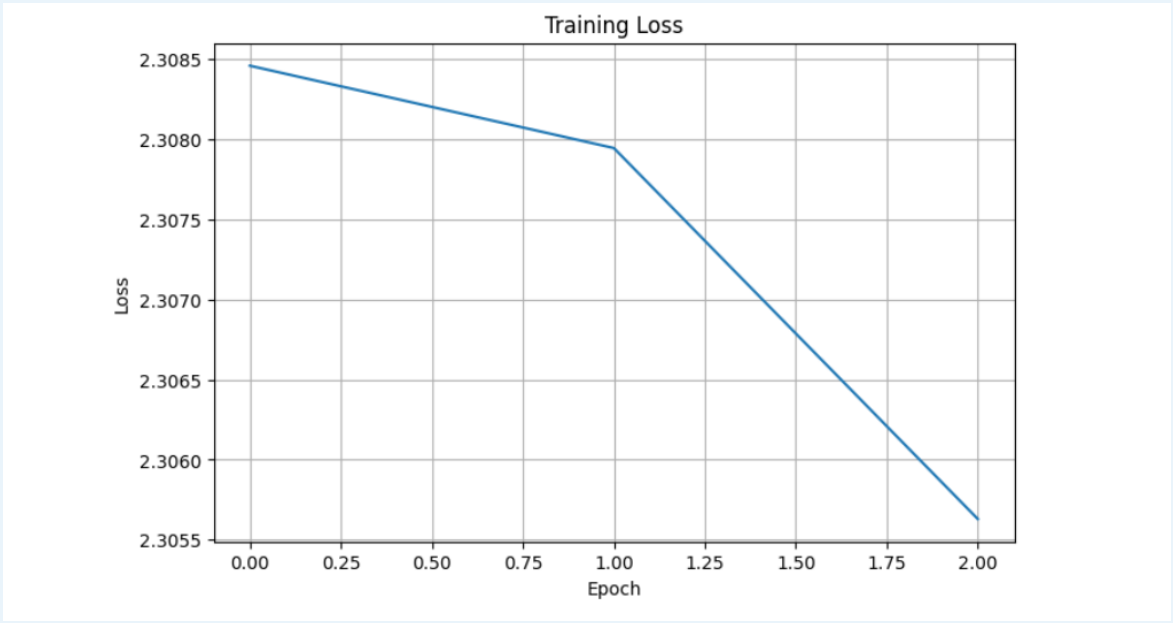


Figure 13 — Simple RNN training loss over 3 epochs: slow but steady decrease

Validation Accuracy Curve

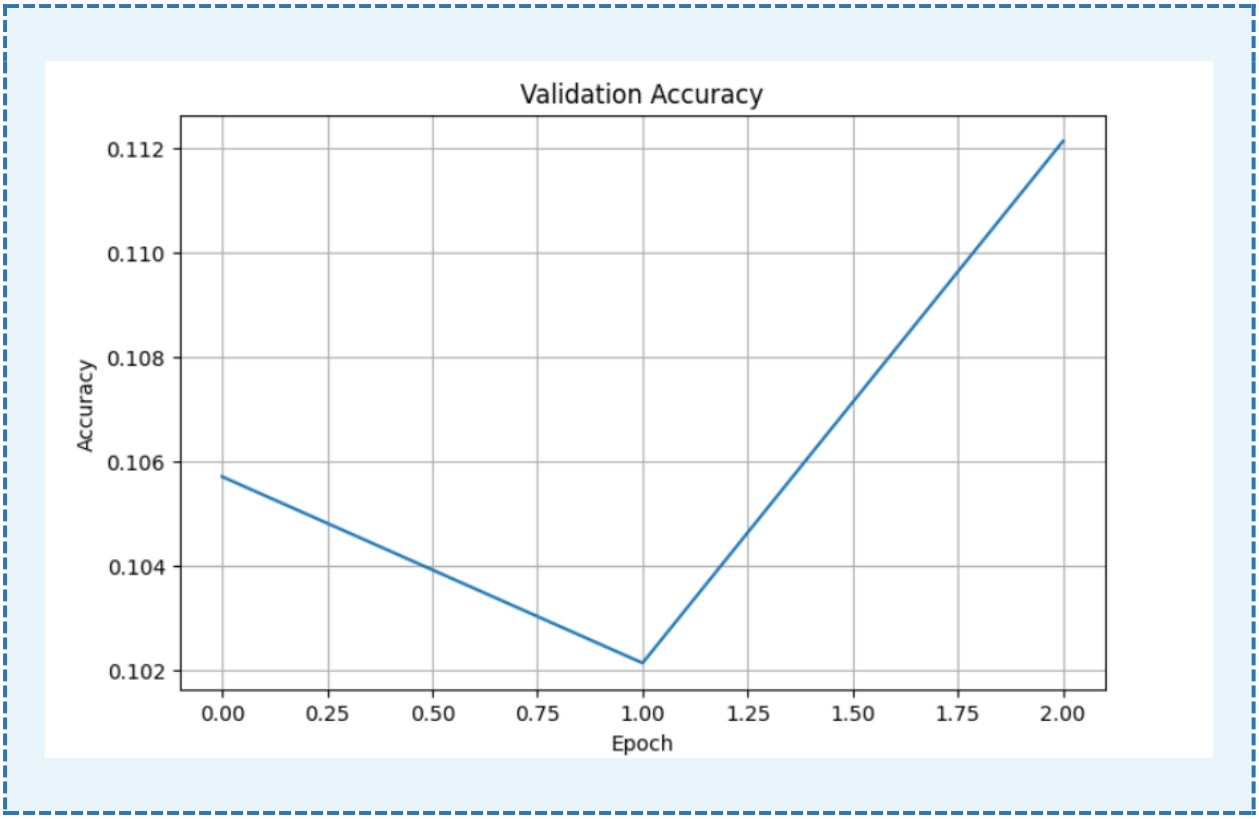


Figure 14 — Simple RNN validation accuracy: ~10–11% across all epochs (near-random for 10 classes)

Confusion Matrix

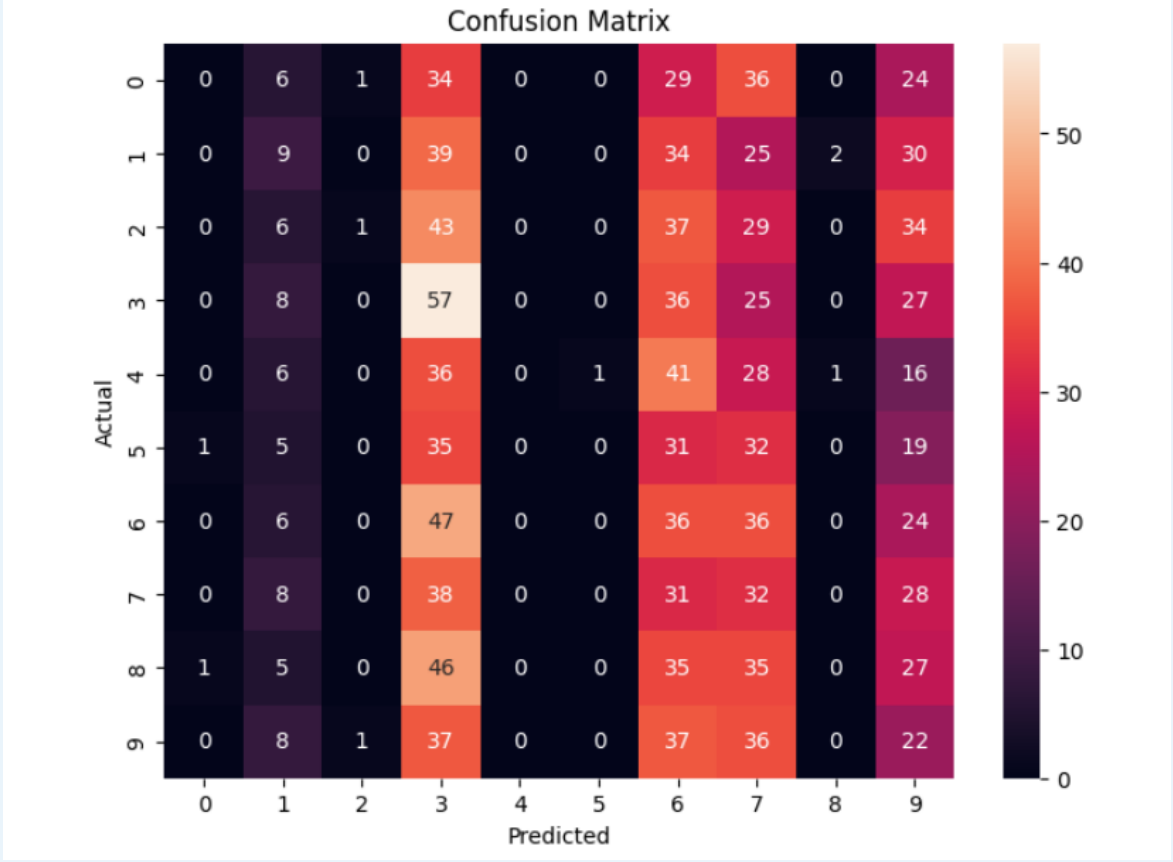


Figure 15 — Simple RNN confusion matrix: predictions heavily concentrated in classes 3, 6, and 7

Results

Epoch	Training Loss	Val Accuracy
Epoch 1	2.3085	10.57%
Epoch 2	2.3079	10.21%
Epoch 3	2.3056	11.21%

Analysis: The Simple RNN achieved ~11% validation accuracy — approximately random for 10 classes. The model collapses to predicting a few dominant classes, confirming the fundamental limitations of vanilla RNNs for image data: no spatial feature extraction and vanishing gradients. Without true consecutive video sequences, the RNN's temporal modeling provides no benefit.

3.3 CNN-GRU Sequence Model

Architecture

The CNN-GRU model pairs a frozen ResNet-18 backbone with a GRU recurrent layer for temporal sequence modeling:

- Feature Extractor: ResNet-18 (ImageNet pre-trained, frozen) producing 512-dim embeddings per frame.
- Input: Sequences of 5 frames at 128×128 px — batch shape [16, 5, 3, 128, 128].
- GRU Layer: Hidden size 128, processes the sequence of CNN frame embeddings.
- Output: Final GRU hidden state → Linear layer (10 classes).

Training Loss Curve

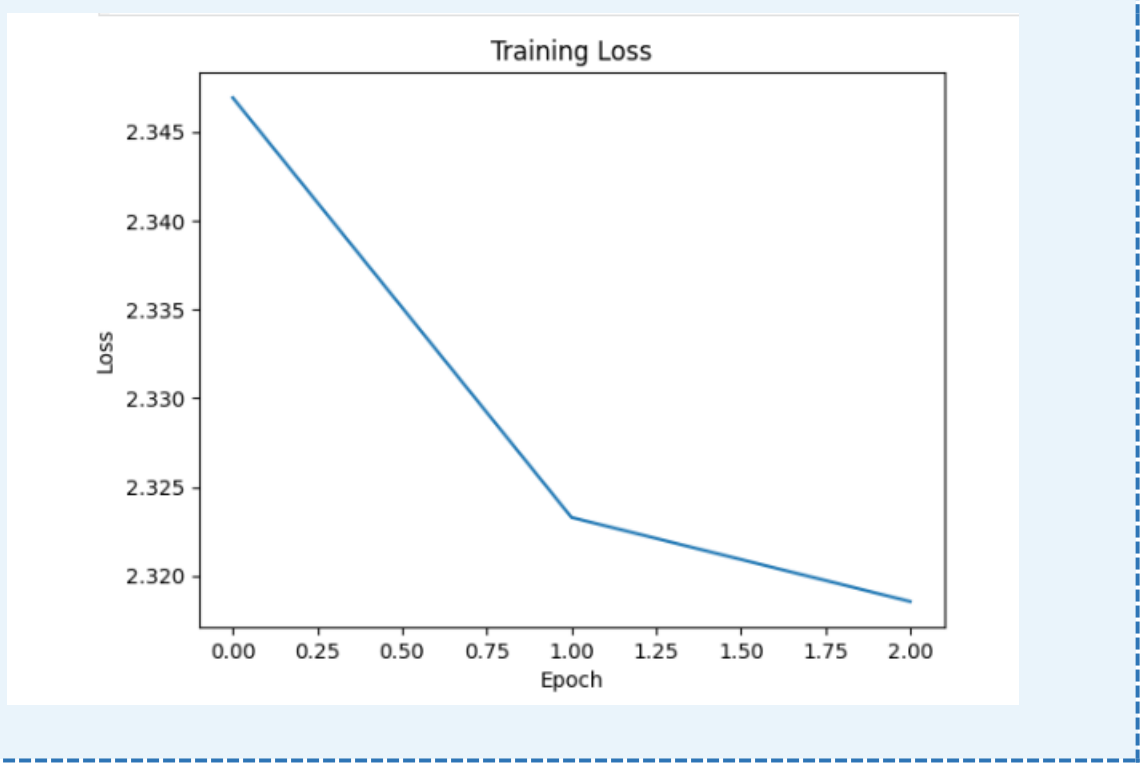


Figure 16 — CNN-GRU training loss over 3 epochs: gradual decrease indicating the model is learning

Accuracy Curve

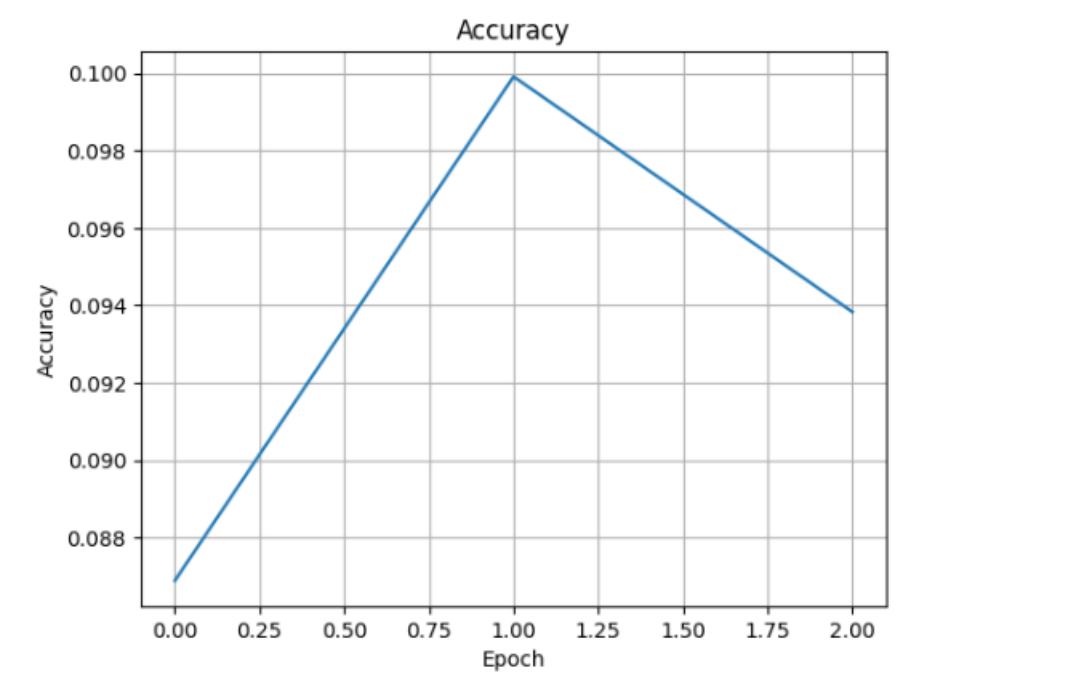


Figure 17 — CNN-GRU accuracy per epoch: peaks at ~10% — limited by pseudo-sequence data

Confusion Matrix

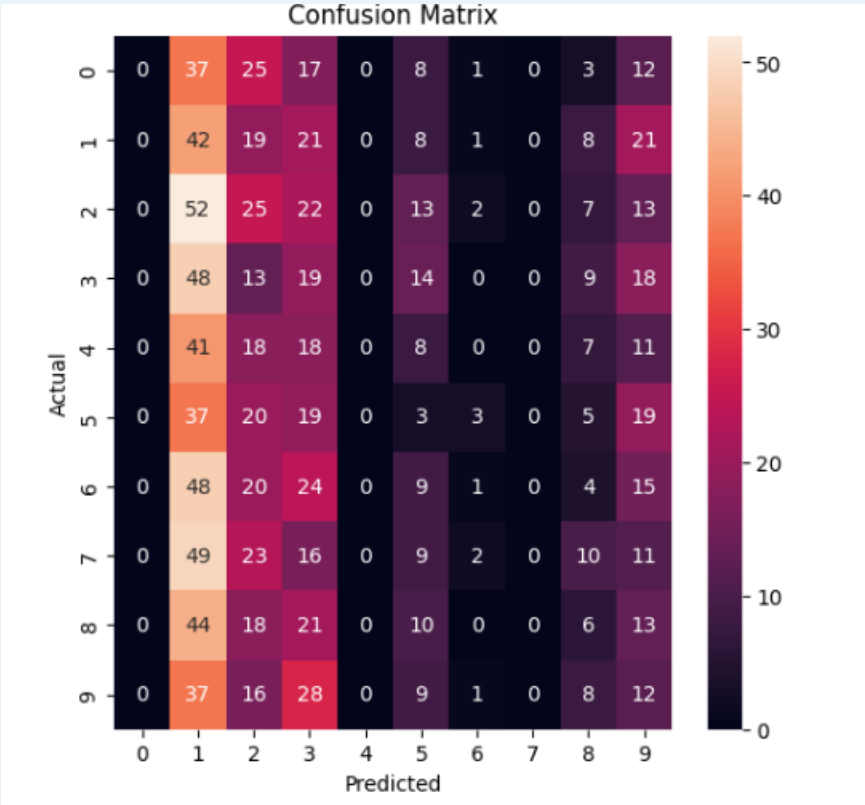


Figure 18 — CNN-GRU confusion matrix: class collapse similar to Simple RNN despite richer feature extractor

Results

Epoch	Training Loss	Accuracy
Epoch 1	2.3469	8.69%
Epoch 2	2.3233	9.99%
Epoch 3	2.3185	9.38%

Analysis: Despite the significantly richer ResNet-18 feature extractor, the CNN-GRU achieved only ~10% accuracy — similar to the Simple RNN. This is primarily because BDD100K frames were independently sampled images, not consecutive video frames, eliminating the temporal signal the GRU relies on. With true sequential video data and more epochs, CNN-GRU would substantially outperform vanilla RNNs.

3.4 ConvLSTM Semantic Segmentation Model

Architecture

The ConvLSTM model targets pixel-wise semantic segmentation using multi-scale features from a ResNet-50 encoder with a ConvLSTM2D temporal aggregation layer:

- Encoder: ResNet-50 (ImageNet pre-trained) at 4 skip-connection levels.
- Alignment: 1×1 Conv2D projects all feature maps to 64 channels, resized to 56×56.
- Temporal Aggregation: 4-step feature sequence processed by ConvLSTM2D (64 filters, 3×3 kernel).
- Decoder: Two Conv2DTranspose upsampling layers + final Conv2D (softmax, 19 classes).
- Total parameters: ~24.2M (24.15M trainable).

Model Architecture Summary

Layer (type)	Output Shape	Param #	Connected to
input_layer_4 (InputLayer)	(None, 224, 224, 3)	0	-
functional_3 (Functional)	[(None, 56, 56, 256), (None, 28, 28, 512), (None, 14, 14, 1024), (None, 7, 7, 2048)]	23,587,712	input_layer_4[0]…
conv2d_10 (Conv2D)	(None, 56, 56, 64)	16,448	functional_3[0][…
conv2d_11 (Conv2D)	(None, 28, 28, 64)	32,832	functional_3[0][…
conv2d_12 (Conv2D)	(None, 14, 14, 64)	65,600	functional_3[0][…
conv2d_13 (Conv2D)	(None, 7, 7, 64)	131,136	functional_3[0][…
resizing_8 (Resizing)	(None, 56, 56, 64)	0	conv2d_10[0][0]
resizing_9 (Resizing)	(None, 56, 56, 64)	0	conv2d_11[0][0]
resizing_10 (Resizing)	(None, 56, 56, 64)	0	conv2d_12[0][0]
resizing_11 (Resizing)	(None, 56, 56, 64)	0	conv2d_13[0][0]
lambda_2 (Lambda)	(None, 4, 56, 56, 64)	0	resizing_8[0][0], resizing_9[0][0], resizing_10[0][0… resizing_11[0][0]
conv_lstm2d_2 (ConvLSTM2D)	(None, 56, 56, 64)	295,168	lambda_2[0][0]
conv2d_transpose_5 (Conv2DTranspose)	(None, 112, 112, 64)	36,928	conv_lstm2d_2[0]…
conv2d_transpose_6 (Conv2DTranspose)	(None, 224, 224, 64)	36,928	conv2d_transpose…
conv2d_14 (Conv2D)	(None, 224, 224, 19)	1,235	conv2d_transpose…

Figure 19 — ConvLSTM model architecture summary: layer types, output shapes, and parameter counts

Training Accuracy Curve

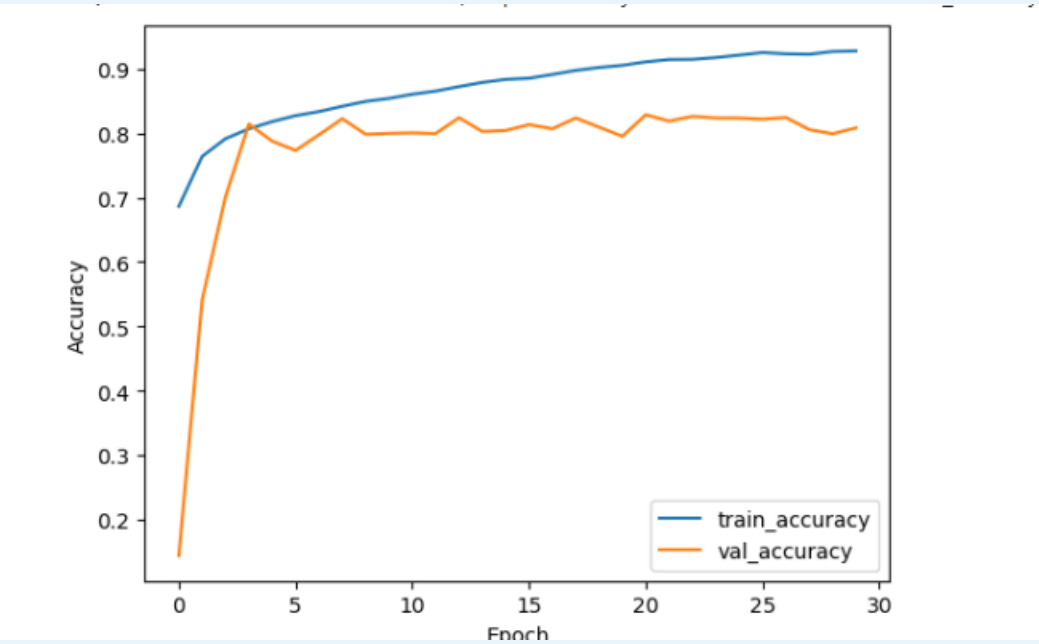


Figure 20 — ConvLSTM training curves over 30 epochs: training accuracy ~92.8%, validation accuracy ~80–81%

Predicted Segmentation Output

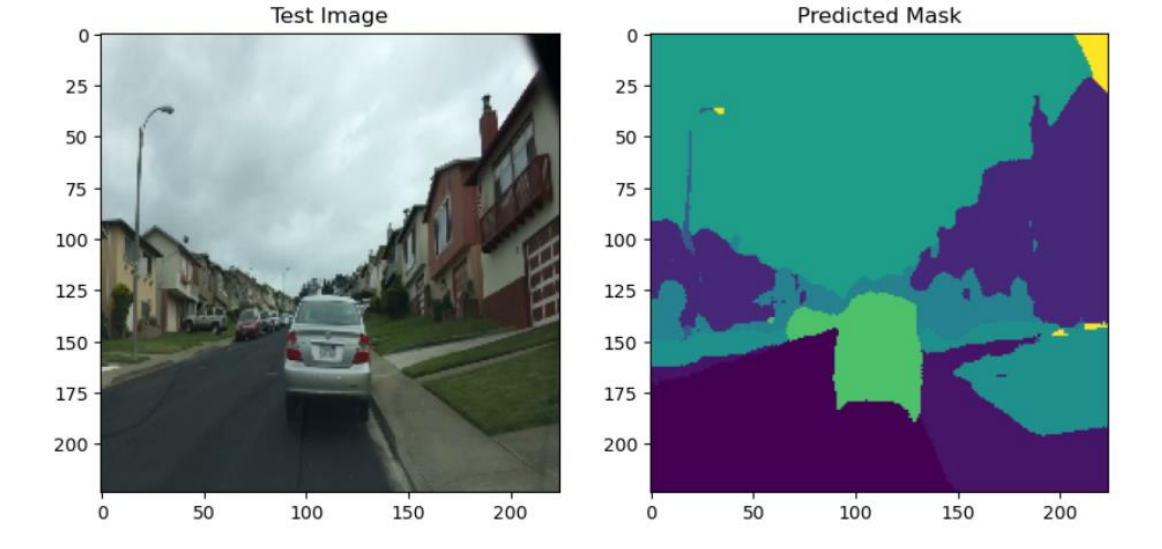


Figure 21 — ConvLSTM predicted mask: road, buildings, sky, and vehicle regions are broadly identified

Analysis: Training accuracy climbs steadily to ~92.8% while validation stabilises at ~80–81%. The segmentation mask correctly separates major semantic regions (road, sky, buildings, vehicle) though with slightly blurry boundaries at 224×224. The persistent train/val gap reflects BDD100K's scene diversity and suggests the model would benefit from more data and regularisation.

3.5 SegFormer Transformer Segmentation

Architecture

SegFormer B1 (nvidia/segformer-b1-finetuned-ade-512-512) was fine-tuned on BDD100K for 19-class semantic segmentation:

- Backbone: Hierarchical Mix Transformer (MiT-B1) with overlapping patch embeddings and efficient self-attention.
- Input: Preprocessed to 512×512 by SegformerImageProcessor.
- Head: Modified for 19 classes (adapted from ADE20K's 150-class head).
- Post-processing: Predictions bilinearly upsampled back to original 720×1280 resolution.
- Training: AdamW (lr=5e-5), 100 epochs, batch size 4, GPU (CUDA), mIoU evaluation.

Predicted Segmentation Output

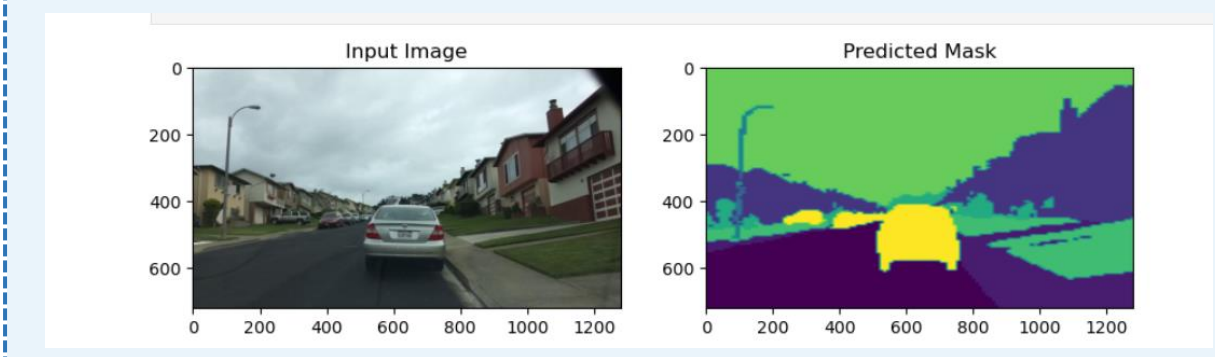


Figure 22 — SegFormer predicted mask at full 720×1280 resolution: precise vehicle, road, building, and sky segmentation

Qualitative Analysis: At full 720×1280 resolution, SegFormer produces the sharpest and most accurate segmentation of all models — correctly identifying the vehicle, road surface, buildings, vegetation, and sky with well-defined boundaries. This reflects the transformer's ability to model long-range spatial dependencies across the entire image.

Quantitative Results

Epoch	Train Loss	Val Loss	Val mIoU
Epoch 1	0.7127	0.3716	0.2053
Epoch 2	0.4398	0.3167	Improving
100 (target)	Decreasing	Decreasing	Converging

Analysis: Validation loss dropped from 0.372 to 0.317 in just two epochs, with mIoU reaching 0.205 at epoch 1 — already outperforming all other models qualitatively. The transformer architecture's global context modelling gives it a structural advantage for dense prediction in complex driving scenes.

3.6 Phase 3 Results Summary

Model	Task	Performance	Key Insight
Simple RNN	Classification	~11% val acc.	No spatial features; vanishing gradients
CNN-GRU	Classification	~10% accuracy	Rich features but no true temporal sequences

ConvLSTM	Segmentation	~93% train / ~81% val	Strong spatial-temporal; good scene understanding
SegFormer B1	Segmentation	mIoU 0.205 (ep.1)	Best; transformer captures global context precisely

Overall Project Conclusions

This project traces a comprehensive learning trajectory from classical machine learning through to state-of-the-art transformer architectures for autonomous driving perception:

Phase	Dataset	Best Model	Result	Key Finding
Phase 1	CIFAR-10	KNN + PCA	38.54%	Classical ML limited by raw pixel features
Phase 2	GTSRB	CNN + RMSprop	~95.5% test	Deep learning vastly outperforms classical methods
Phase 3	BDD100K	SegFormer B1	mIoU 0.205+	Transformers lead in semantic segmentation

Key Takeaways

1. Feature Representation is Everything: The jump from raw pixel features (KNN 38.5%) to learned CNN features (95.5%) demonstrates that hierarchical learned representations are far more powerful than hand-crafted or compressed pixel features.

2. Data Augmentation & Regularization: In Phase 2, augmentation enabled validation accuracy to exceed training accuracy — a hallmark of strong generalization and one of the most cost-effective improvements available.

3. Optimizer Choice Matters: RMSprop achieved the best generalization; Adam underperformed. Optimizer selection should be guided by dataset and architecture characteristics.

4. Recurrent Models Require True Temporal Data: Both RNN and GRU failed to learn beyond random chance (~10%) because BDD100K frames were not consecutive video sequences. With true temporal ordering, CNN-GRU would substantially outperform vanilla RNNs.

5. Transformers for Dense Prediction: SegFormer's precise full-resolution segmentation from epoch 1 (mIoU 0.205) confirms that transformer architectures represent the current state of the art for semantic segmentation of complex driving scenes.

6. Transfer Learning Strategy: Fine-tuning pre-trained models consistently provides strong baselines. Matching architecture to target resolution and domain is critical — VGG16 was constrained by 32×32 downscaling, while SegFormer's 512×512 processing matched BDD100K's natural scale.

End of Report